

UK PATENT APPLICATION

SYSTEM AND METHOD FOR AUTONOMOUS CROSS-DOMAIN CAUSAL DISCOVERY AND BUSINESS GENERATION USING ENSEMBLE PREDICTION AND TEST-DRIVEN DEVELOPMENT

DRAWINGS INFORMATION

Overview

The present patent application includes sixteen (16) figures illustrating the system architecture, data flow, algorithmic processes, and operational timeline of the autonomous cross-domain causal discovery and business generation system. Each figure is described below with reference to the relevant claims and identification of the novel technical contributions (Innovations (1)-(7)).

Figure Descriptions

Figure 1: System Architecture Overview

System architecture diagram showing the complete nine-stage autonomous pipeline comprising: (1) multi-source data ingestion, (2) time-series standardisation, (3) NxN cross-domain correlation computation, (4) bidirectional Granger causality testing, (5) three-model ensemble prediction generation, (6) exploitation recommendation scoring, (7) autonomous TDD code generation, (8) deployment automation, and (9) outcome measurement with weight recalibration. Data flow arrows connect each stage sequentially with a feedback arrow from stage 9 back to stage 5 indicating the closed-loop recalibration mechanism. Per Claims 1, 11, 16. Innovation (4) (Closed-Loop Autonomous Business Pipeline).

Figure 2: Multi-Source Data Ingestion Pipeline

Data ingestion pipeline diagram showing six external data sources grouped into two layers: Layer 1 (Wikipedia, Reddit) labelled as "Behavioural Signals" and Layer 2 (FRED, Stock, ArXiv, GDELT) labelled as "Exploitable Outcomes." Each source connects through a standardisation module that produces unified TimeSeriesData records comprising variable_id, source tag, timestamp, and numerical value, stored in a PostgreSQL database. The VariableMetadata table is shown as a reference table linking variable names to source identifiers. Per Claims 1, 2, 11. Innovation (1) (Variable-Level Cross-Domain Causal Discovery).

Figure 3: Cross-Domain Correlation Engine

Correlation analysis diagram showing an NxN variable matrix (upper triangle only) with three correlation computation blocks (Pearson, Spearman, Kendall) feeding into a significance filter ($p < 0.05$) and a cross-domain enforcement filter that excludes same-source pairs. Output arrows lead to the CorrelationResult table. A callout box contrasts the invention's variable-level approach with prior art domain-level correlation. Per Claims 1, 3, 11. Innovations (1) (variable-level discovery) and (5) (cross-layer enforcement).

Figure 4: Bidirectional Granger Causality System

Granger causality testing diagram showing two parallel test paths: Forward Test (H_0 : X does not Granger-cause Y) and Reverse Test (H_0 : Y does not Granger-cause X). A frequency-adaptive lag selector feeds maximum lag values (252/52/12/4/2) based on data frequency. The two test results feed into a direction assignment decision block with four outputs: x_to_y, y_to_x, bidirectional, none. A minimum requirements box specifies ≥ 30 data points and ≥ 5 period buffer. Per Claims 1, 4, 11, 34. Innovation (1) (causal direction determination).

Figure 5: Three-Model Ensemble Architecture

Ensemble prediction diagram showing three sub-model blocks: Granger Predictor (weight 0.40, confidence from $1-p_value$), OLS Predictor (weight 0.35, confidence from R^2 , min 6 months training), and ARIMA(1,1,1) Predictor (weight 0.25, min 24 data points). Each sub-model outputs direction and confidence. A feature engineering block (rolling windows 7, 30, 90) feeds into the sub-models. The three outputs feed into a weighted voting combiner showing the formulas: $up_score = \sum(w_i \times c_i)$ for "up" models, $down_score = \sum(w_i \times c_i)$ for "down" models. A calibration feedback arrow from the PredictionTracking table (≥ 10 validated predictions required) feeds updated weights back to the combiner. Per Claims 1, 5, 6, 11, 12, 21, 24. Innovation (2) (Three-Model Ensemble with Automatic Weight Calibration).

Figure 6: Cross-Layer Enforcement

Architectural constraint diagram showing Layer 1 (Wikipedia, Reddit) on the left and Layer 2 (FRED, Stock, ArXiv, GDELT) on the right, connected by directional arrows labelled "Signal -> Target" flowing exclusively from Layer 1 to Layer 2. A central box shows the SQL-level filter: "cross_layer = OR(AND(var1 IN L1, var2 NOT IN L1), AND(var2 IN L1, var1 NOT IN L1))". Crossed-out arrows between Layer 1 variables (Wikipedia<->Reddit) and between Layer 2 variables (FRED<->Stock) indicate prohibited same-layer predictions. Per Claims 1, 7, 11, 20. Innovation (5) (Cross-Layer Enforcement Architecture).

Figure 7: Exploitation Scoring Pipeline

Exploitation scoring diagram showing five input components with their weights: Demand (30), Growth (25), Timing (20), Evidence (15), Competition (10), feeding into a summation block that produces opportunity_score (0-100). An action type classifier block below receives the opportunity_score and build_viability_score, producing one of four action types: BUY, SELL, BUILD, MONITOR. The BUILD output connects via arrow to the TDD pipeline (Figure 8). A backtester feedback block shows the actual vs. predicted comparison formula: $actual_pct = ((outcome - baseline) / |baseline|) \times 100$. Per Claims 1,

8, 11, 17, 26. Innovations (2) (scoring) and (4) (autonomous action).

Figure 8: Autonomous TDD Code Generation Overview

High-level code generation pipeline diagram showing an exploitation recommendation input flowing through five sequential phases: SPEC (YAML generation), RED (failing tests), GREEN (AI implementation), REFACTOR (quality improvement), VALIDATE (final checks). An AI language model icon connects to RED, GREEN, and REFACTOR phases. The GREEN phase includes a decision diamond: "pass_rate >= 80%?" with Yes leading to REFACTOR and No leading to a retry loop (max 5 attempts). Output branches to LIVE (green) or FAILED (red) with error_breakdown JSON. Per Claims 1, 9, 11, 13, 16, 28. Innovation (3) (Autonomous TDD-Driven MVP Generation).

Figure 9: Five-Phase TDD Pipeline Detail

Detailed pipeline diagram expanding each of the five phases: SPEC phase shows keyword analysis across eight categories (frontend, backend, database, auth, deployment, payments, analytics, CRUD) feeding template matching (layer alignment 30pts, tag 15pts, framework 10pts, description overlap 5pts) producing YAML with complexity tiers (LOW: 4 AC, MEDIUM: 7 AC, HIGH: 12 AC). RED phase shows test file generation and stub module creation with NotImplementedError. GREEN phase shows symbol extraction via regex, AI submission with failure context, pass rate computation formula, and five-retry loop. REFACTOR phase shows AI refinement with regression guard. VALIDATE phase shows AST parsing, py_compile, final pytest. Per Claims 9, 10, 13, 14, 22, 28. Innovation (3) (TDD pipeline specifics).

Figure 10: Deployment Automation Flow

Deployment automation diagram showing two parallel tracks: GitHub track (repository creation via PyGithub with auto_init=True -> file push via Git Tree API with InputGitTreeElement mode "100644" -> commit creation -> SHA verification) and Railway track (projectCreate GraphQL mutation -> serviceCreate mutation -> serviceInstanceDeploy mutation -> status polling loop). Build status state machine shown: QUEUED -> GENERATING -> LIVE or FAILED. A stale-build detector monitors all active states with 600-second timeout. Per Claims 1, 9, 11, 15, 16, 33. Innovations (3) (autonomous deployment) and (4) (no human intervention).

Figure 11: Outcome Measurement and Feedback Loop

Outcome measurement diagram showing three measurement tracks: (1) Prediction Validation - daily at 02:30 UTC, querying pending predictions where target_date <= today, computing direction_correct and actual_change_pct, transitioning status from 'pending' to 'validated'. (2) Walk-Forward Backtesting - weekly on Monday at 04:00 UTC, expanding-window OLS from split index 6, recording predictions with model_version='backtest_walkforward'. (3) Exploitation Backtesting - measuring actual target growth over opportunity window, computing direction accuracy and mean absolute error. All three tracks feed into a weight recalibration block that updates ensemble sub-model weights and exploitation component weights. Per Claims 1, 5, 6, 11, 12, 16, 17, 25. Innovations (2) (weight calibration) and (4) (closed-loop validation).

Figure 12: Build-Iterate Loop

Build iteration diagram showing a failed build entering a failure diagnosis module that classifies errors across seven categories (syntax, test, frontend, import, wiring, config, runtime) represented as a stacked bar chart. The diagnosis feeds into an iteration context generator that collects three inputs: parent error_breakdown (JSON), parent generated files, and current ensemble predictions. The context feeds into a child build spawner that creates a new MVPBuild record with parent_build_id linkage, iteration_number + 1, and iterate_reason. A concurrency guard gate checks for active builds (QUEUED/GENERATING) in the chain before allowing spawning. The child build re-enters the five-phase TDD pipeline. An iteration chain timeline shows: Build v1 (FAILED) -> Build v2 (FAILED) -> Build v3 (LIVE), illustrating progressive self-correction. Per Claims 28, 29, 30, 31. Innovation (6) (Build-Iterate Loop with Failure Diagnosis).

Figure 13: Autonomous Scheduling Timeline

Timeline diagram showing a 24-hour UTC clock with cron jobs marked at their scheduled times: 01:00 (Wikipedia ingestion, daily), 01:30 (Reddit ingestion, daily), 02:00 (GDELT ingestion, daily), 02:30 (prediction actuals update, daily), 03:00 (baseline snapshot, weekly Sunday), 04:00 (walk-forward backtest, weekly Monday), 05:00 (ensemble predictions, daily). Data dependency arrows between jobs show: ingestion (01:00-02:00) feeds correlation/Granger analysis, actuals update (02:30) feeds weight calibration, which feeds ensemble predictions (05:00). A thirty-minute spacing annotation indicates API rate limit prevention. Per Claims 1, 11, 16, 23, 27. Innovation (4) (autonomous scheduling).

Figure 14: End-to-End Example Scenario

Complete walkthrough diagram showing nine numbered steps in sequence: (1) Wikipedia "Artificial_intelligence" pageviews ingested as Layer 1 variable, (2) Correlation $r=0.72$ with NASDAQ (Layer 2), (3) Granger test: forward $p=0.03$ at lag 3 (significant), reverse $p=0.28$ (not significant), direction= x_to_y , (4) Ensemble prediction: Granger up/0.97 + OLS up/0.68 + ARIMA up/0.55 = combined up/0.765, (5) Exploitation score: 72 (demand 25.5 + growth 15.0 + timing 14.0 + evidence 14.55 + competition 3.0), action=BUILD, (6) TDD pipeline: 7 acceptance criteria (MEDIUM), 92% pass rate, (7) GitHub push + Railway deploy -> LIVE URL, (8) Three-month validation: predicted "up" vs actual movement, direction_correct computed, Granger weight updated, (9) Commercial intelligence: Stripe webhook aggregates revenue events into ProductMetrics, GA4 pull records engagement, composite score 78/100 computed, winning configuration (React+freemium+technology) identified, confidence 82% injected into next specification generation. All seven innovation markers ((1)(2)(3)(4)(5)(6)(7)) are annotated at the relevant steps. Per Claims 1, 11, 16, 35.

Figure 15: Commercial Intelligence Architecture

Commercial intelligence system architecture diagram showing three input pipelines and four output consumers. Input Pipeline 1 (Revenue): Stripe webhook events (checkout.session.completed, customer.subscription.updated, customer.subscription.deleted, invoice.payment_succeeded) flow through a RevenueEvent logger and then into an aggregate_revenue_to_metrics() processor that upserts ProductMetrics records with mrr_cents, subscriber_count, and churn_rate per monthly period. Input Pipeline 2 (Engagement): A daily scheduled job invokes the GA4 BetaAnalyticsDataClient with a RunReportRequest for page_views, unique_visitors, and

avg_session_duration filtered by deployment hostname, storing results in ProductMetrics with source="ga4". Input Pipeline 3 (Deployment Tracking): The MVPBuilder build completion handler creates a ProductDeployment record within the same database transaction, linking build_id, recommendation_id, tech_stack snapshot, pricing_model, market_category, and target_demographic. The three inputs converge into a Commercial Intelligence Engine containing four processing blocks: (A) Commercial Ranking - normalises revenue, engagement, conversion, and retention to 0-100 scale and computes $\text{composite_score} = 0.40 \times \text{revenue} + 0.30 \times \text{engagement} + 0.20 \times \text{conversion} + 0.10 \times \text{retention}$; (B) Winning Configuration Analyser - groups by tech_stack, pricing_model, market_category, target_demographic and computes per-group average scores; (C) Configuration Confidence Scorer - matches proposed configuration dimensions against winning configurations to produce 0-100 confidence with per-dimension breakdown; (D) Demographic Pattern Mapper - groups by target demographic segment and identifies most common successful configs. Output Consumer 1: Dashboard Commercial Case panel (confidence %, progress bar, recommendation text). Output Consumer 2: Specification Generator (commercial context injected into AI prompt). Output Consumer 3: REST API endpoints (/api/commercial-intelligence/*). Output Consumer 4: Portfolio table (live engagement and revenue_mrr columns). Per Claims 35, 36, 37, 38, 39, 40. Innovation (7) (ML-Driven Commercial Intelligence Feedback Loop).

Figure 16: Demographic Pattern Mapping and Confidence Scoring

Dual-panel diagram. Left panel (Demographic Pattern Mapping): A table with columns [Demographic Segment | Deployment Count | Avg Score | Top Stack | Top Pricing | Top Market]. Example rows: "18-25 tech enthusiasts" (12 deployments, avg 71, React, freemium, technology), "35-50 professionals" (8 deployments, avg 64, Next.js, subscription, finance), "25-35 urban millennials" (6 deployments, avg 58, React, freemium, lifestyle). Arrows from the table lead to a "Segment Insights Generator" that produces plain-English recommendations (e.g., "For tech enthusiasts: React + freemium performs 22% above average"). Right panel (Configuration Confidence Scoring): A flowchart showing a proposed configuration input {tech_stack: "Vue.js", pricing: "one-time", market: "health", demographic: "seniors"} entering a four-step scoring pipeline. Step 1: For each dimension, query winning configurations database for matching entries. Step 2: If match found, confidence = historical_avg_score + min(5 x deployment_count, 20); if no match, confidence = 30 (baseline). Step 3: Average all four dimension confidences to produce overall score. Step 4: Classification - ≥ 75 : "High confidence - strong historical support", 50-74: "Moderate confidence - some supporting data", < 50 : "Low confidence - limited precedent, consider alternatives". A colour-coded output bar shows the overall score with green/amber/red zones. Per Claims 38, 39, 40. Innovation (7).

Drawing Format

All figures shall be prepared as formal patent drawings in black and white line art format, suitable for reproduction at two-thirds scale. Flow diagrams use rectangular boxes for processes, diamond shapes for decision points, cylindrical shapes for databases, and directional arrows for data flow. Innovation markers ((1)-(7)) are placed adjacent to the relevant process blocks as circled numerals.